

Auditoria e Blueprint do LF ERP para Android

Levantamento linha a linha do backend, frontend e banco de dados reais do LF ERP, seguido do mapa funcional completo, da tecnologia recomendada e da arquitetura proposta para um cliente Android nativo que compartilha a mesma API, o mesmo banco e as mesmas regras de negócio da versão web.

Data: 08/09/2026 Escopo: backend/ + frontend/ (repositórios completos)

Arquivos de rota analisados: 48 Migrations analisadas: 41

Módulos JS de frontend: 44 Endpoints mapeados: ~340

- **Nenhuma tela foi criada.**

Conforme solicitado, esta é a primeira entrega — auditoria e arquitetura apenas. Implementação aguarda aprovação explícita deste documento.

O LF ERP já está pronto para virar um app — a base é sólida

O LF ERP atual é muito maior do que a documentação interna (CLAUDE.md) sugere: não são 9 módulos, são **cerca de 35 domínios funcionais** — de PDV e financeiro a NF-e, marketplace, CRM, fidelidade, WhatsApp e um painel SaaS multiempresa completo — servidos por **48 arquivos de rota** e **≈340 endpoints REST** sobre **PostgreSQL (Neon)**, com **41 migrations** aplicadas e um modelo multiempresa (empresa/empresa_id) já consolidado.

Três achados mudam o cálculo do projeto Android:

1. **A API não precisa ser recriada.** Ela já devolve JSON, já aplica JWT + validarAcessoEmpresa + permissões por módulo/ação em praticamente todas as rotas, e não depende de sessão de navegador.
2. **A venda do PDV já roda em transação única** (venda → itens → baixa de estoque → contas a receber) e já possui idempotency_key com índice único no banco — exatamente o mecanismo que uma fila offline no Android precisa para reenviar uma venda sem duplicar.
3. **O PDV web já resolve offline de verdade** (fila em IndexedDB com sincronização ao reconectar), então o Android não está inventando esse conceito — está portando um padrão já validado em produção para Room + WorkManager.

A recomendação de tecnologia é **Kotlin + Jetpack Compose nativo** (detalhada na Seção D) — não WebView, não híbrido. O restante deste documento cobre a auditoria completa, o inventário funcional, o mapa de paridade Web→Android, a arquitetura, a estratégia de offline/segurança/testes e o roadmap em fases.

A Auditoria do sistema real

Produzida por leitura direta de 100% dos arquivos de rota, todas as 41 migrations e todos os módulos de frontend — nenhuma suposição, apenas o que o código faz hoje.

A.1 — Estrutura do projeto

O diretório `lf-erp/` não é, ele próprio, um repositório Git — mas `backend/` e `frontend/` são **dois repositórios Git independentes** lado a lado (cada um com seu próprio `.git/`), cada um com deploy próprio (Render e Vercel, respectivamente). Isso importa para o app Android: ele deve nascer como um **terceiro repositório irmão** (ex.: `lf-erp-android/`), nunca dentro de `backend/` ou `frontend/`.

Backend: 48 arquivos em `backend/routes/`, 21 em `backend/utils/`, 41 arquivos SQL em `backend/migrations/`, um único `server.js` de composição (655 linhas) e um `backend/db/initDb.js` legado. Frontend: 44 módulos em `frontend/js/`, mais 5 páginas HTML independentes (`index.html`, `admin.html`, `checkout.html`, `etiquetas.html`, `portal.html`, `orcamento-pdf.html`).

A.2 — Backend

`server.js` é um *composition root*: cada arquivo de rota exporta uma factory que recebe `{pool, auth, writeRateLimiter, ...helpers}` e devolve um Router — as rotas não importam o pool nem o middleware de autenticação sozinhas. Isso deixa qualquer camada nova (como uma API mobile) sem trabalho extra: o pool de conexões, a autenticação e as permissões já são objetos compartilhados, não algo amarrado ao Express de um jeito que precise ser refeito.

Não existe `db.js` — o Pool do pg é criado diretamente em `server.js` (máx. 10 conexões, SSL obrigatório para hosts `neon.tech`). Segurança HTTP é resolvida por um middleware próprio (headers manuais tipo helmet), não pelo pacote helmet. O processo falha rápido (`process.exit(1)`) se `JWT_SECRET` ou `DATABASE_URL` estiverem ausentes.

Achado relevante: o schema é mantido por *dois* mecanismos em paralelo — `initDb.js` (bootstrap idempotente, hoje um no-op em produção porque a tabela `empresas` já existe) e as 41 migrations numeradas em `migrations/`,

aplicadas a cada boot via `runner.js` e rastreadas numa tabela `_migrations`. Isso não bloqueia o Android, mas é uma dívida técnica a observar caso alguém precise recriar o banco do zero.

A.3 — Frontend

É uma SPA sem framework: um único `#app` em `index.html` alterna entre login/registro/app principal trocando classes `hidden`; a navegação lateral usa `data-view` e carrega cada módulo sob demanda via `import()` dinâmico — já há *code-splitting* por módulo (≈ 28 imports dinâmicos). Já é um PWA instalável (`manifest.json` + `sw.js`), mas o service worker só cacheia o "shell" da aplicação — a API nunca passa pelo cache do SW.

O PDV (`pdv.js` + `pdvOffline.js`) é o único módulo com offline real: cache de produtos/clientes em IndexedDB e uma fila de vendas pendentes que sincroniza sozinha ao voltar a conexão, reaproveitando a `idempotency_key` do backend. Nenhum outro módulo tem suporte offline.

`admin.html` é o painel SaaS (dono da plataforma, todas as empresas) — fisicamente separado da SPA, com seu próprio JavaScript duplicando parte da lógica de `api.js`. `portal.html` é o autoatendimento do cliente final, com um JWT completamente diferente (`authCliente`).

Não existe qualquer leitura de código de barras por câmera hoje — `codigo_barras` é só um campo de texto (digitado ou vindo de leitor USB/HID). O único uso de "barcode" no frontend é `etiquetas.js`, que *imprime* códigos Code 39 via fonte CSS — geração, não leitura. Isso confirma que a leitura por câmera é uma capacidade 100% nova a construir no Android (Seção H).

A.4 — Banco de dados (PostgreSQL / Neon)

Mais de 80 tabelas, agrupadas por domínio: usuários/empresas/permissões, produtos/estoque (incluindo grades, kits, depósitos, lotes/séries), vendas/PDV (incluindo caixa, orçamentos, pedidos, devoluções, comissões, metas), financeiro (contas a pagar/receber, lançamentos, conciliação, investimentos), fiscal (NF-e/NFC-e/NFS-e), e CRM/fidelidade/marketplace/integrações.

Nenhuma das tabelas transacionais centrais (`vendas`, `venda_itens`, `compras`, `compra_itens`, `contas_receber`, `contas_pagar`) declara *foreign keys* reais — a integridade referencial é só de aplicação. Isso já é assim hoje na Web; o Android não piora nem resolve esse ponto, apenas herda o mesmo contrato.

ACHADO-CHAVE PARA O OFFLINE DO ANDROID

vendas e compras já têm uma coluna `idempotency_key` com **índice único parcial** por empresa (`migration 041` e `initDb.js`). Uma venda reenviada com a mesma chave é automaticamente deduplicada pelo Postgres — é exatamente o contrato que uma fila offline no Android precisa, e já existe, testado em produção pelo PDV web.

Nenhuma outra tabela (itens de venda, movimentações de estoque, contas a receber avulsas, devoluções) tem chave de idempotência própria — o que é seguro porque essas linhas nascem dentro da mesma transação da venda/compra "pai", nunca são inseridas isoladamente pelo cliente.

A.5 — API

REST sobre JSON, com envelope padrão `{"sucesso": bool, "dados": ..., "erro": "..."}`. Três superfícies de autenticação coexistem:

- **JWT de staff** (`Authorization: Bearer`) — usado por toda a SPA e é o que o app Android deve reutilizar.
- **JWT de cliente** (`authCliente`) — exclusivo do portal do cliente final, fora de escopo do app Android de gestão.
- **X-API-Key** — API pública (`/api/v1/*`) pensada para integrações de terceiros, não para o app oficial.

O inventário completo, endpoint a endpoint, está na Seção B.

A.6 — Autenticação

JWT HS256, expira em 12h, claims incluem `id`, `tipo`, `is_saas_owner`, `empresa`, `empresa_id`, `empresa_nome`, `plano_codigo`, `assinatura_status`. Logout gera uma blacklist (tabela `jwt_blacklist` + cache em memória). POST `/auth/refresh` exige um token ainda válido e assina um novo — não existe um refresh token de longa duração separado nem rotação. Senhas legadas em texto puro são migradas automaticamente para bcrypt no primeiro login bem-sucedido. Login tem rate limit por IP e por usuário.

A.7 — Multiempresa

Padrão duplo confirmado: `empresa` (TEXT, legado) e `empresa_id` (INTEGER). Desde a `migration 036`, `empresa_id` é NOT NULL em todas as tabelas centrais — mas `empresa` nunca foi removida, exatamente como exige a regra de retrocompatibilidade do projeto. A função `validarAcessoEmpresa()`

(em `utils/empresa.js`) é o portão único: resolve a empresa alvo e confirma que bate com o `empresa_id` ou o nome do token — só o dono da plataforma (`is_saas_owner`) escapa dessa checagem, por desenho (painel SaaS). O filtro SQL padrão, repetido em quase toda rota, é `(empresa_id = $1 OR (empresa_id IS NULL AND empresa = $2))`.

A.8 — Permissões

Modelo de duas camadas: `permissoes_padrao` (default por `tipo_usuario`: `admin/gerente/funcionário`) e `permissoes_usuario` (override por usuário + empresa). 38 módulos × 11 ações possíveis (`ver`, `criar`, `editar`, `deletar`, `configurar`, `emitir`, `cancelar`, `acumular`, `resgatar`, `ajustar`, `excluir`). `requirePermissao(pool, modulo, acao)` valida contra uma lista branca antes de montar SQL. Paralelamente, várias rotas usam apenas checagens grosseiras por tipo (`podeGerenciarFinanceiro/Compras/Usuarios/Vendas`) — as duas camadas convivem, e o Android precisa respeitar ambas exatamente como a Web já respeita.

A.9 — Módulos identificados

OPERAÇÃO

Dashboard · Produtos · Grades · Kits ·
Imagens · Tabelas de Preço · Categorias ·
Estoque · Depósitos · Rastreabilidade
(lote/série) · Etiquetas

COMERCIAL

Vendas · PDV · PDV Offline · Devoluções ·
Caixa · Orçamentos · Pedidos · Comissões ·
Metas · CRM

CADASTROS

Clientes · Portal do Cliente · Fornecedores ·
Compras · Filiais · Usuários/Permissões

FINANCEIRO

Contas a Receber · Contas a Pagar ·
Lançamentos · Fluxo de Caixa ·
Investimentos · Conciliação Bancária · Caixa
(sessões)

RELATÓRIOS/BI

DRE · Lucratividade · Inadimplência · Vendas
por Grade · BI executivo (com IA) ·
Exportação Contábil

FISCAL & PAGAMENTOS

NF-e · NFC-e · NFS-e · Checkout/Links · PIX
(EFÍ) · Boleto (Asaas)

INTEGRAÇÕES

Marketplace (Mercado Livre) · WhatsApp ·
Webhooks/API Pública · Fidelidade · Alertas
de Cobrança · Notificações/SSE

PLATAFORMA (SAAS)

Admin SaaS (todas as empresas) ·
Configurações · Lixeira

A.10 — Dependências

Backend: pg, jsonwebtoken, bcrypt, express, cors, cloudinary, nodemailer, multer, @sentry/node, @anthropic-ai/sdk (usado no insight de IA do BI, modelo claude-haiku-4-5). Sem axios (usa fetch nativo do Node 18). Sem biblioteca de PDF — os documentos fiscais vêm prontos da Focus NFe.

Frontend: zero framework, zero bundler. Chart.js, Font Awesome e Google Fonts via CDN, Sentry Browser SDK, e uma fonte CSS ("Libre Barcode 39 Text") só para imprimir etiquetas.

Achados classificados

Problemas reais encontrados durante a leitura do código — reportados, não corrigidos, conforme a Regra da Seção 59. Nenhum bloqueia o início do projeto Android; todos afetam Web e Android igualmente, por serem no backend compartilhado.

ALTO

Três implementações independentes do "fluxo de caixa"

O mesmo cálculo de entradas/saídas está reimplementado separadamente em `fluxo-caixa.routes.js`, `financeiro.routes.js` e `relatorios.routes.js`, cada um com pequenas diferenças de limite/formatação. Risco de os três divergirem silenciosamente com o tempo.

ARQUIVOS	<code>fluxo-caixa.routes.js</code> , <code>financeiro.routes.js</code> , <code>relatorios.routes.js</code>
IMPACTO	Manutenção; risco de números de fluxo de caixa divergentes entre telas.
AFETA WEB?	Sim, já hoje.
AFETA ANDROID?	Sim — recomenda-se escolher uma única rota como fonte de verdade ao integrar o app.

ALTO

Possível referência não importada em `admin.routes.js`

Funções como `verificarWebhookAsaas`, `addDias`, `consultarBoletoAsaas` e `obterPlanoEmpresa` são usadas no arquivo sem aparecer nos imports/parâmetros da `factory` — pode ser injeção implícita válida ou um `ReferenceError` latente em produção, especialmente no webhook de assinatura SaaS. Não confirmado sem rodar o endpoint; merece verificação isolada antes de qualquer mudança nessa área.

ARQUIVO	<code>backend/routes/admin.routes.js</code>
LOCAL	POST <code>/admin/billing/webhook-assinatura</code> e rotas de billing correlatas
RISCO	Falha silenciosa na confirmação automática de pagamento de assinatura SaaS.
AFETA ANDROID?	Não diretamente — é rota de operação da plataforma, fora do escopo do app.

ALTO**hoje() possivelmente fora de escopo em exportação LGPD**

O endpoint de exportação completa de dados da empresa usa `hoje()` no nome do arquivo gerado sem que a função apareça entre as dependências injetadas no módulo — suspeita de erro em tempo de execução ao usar esse recurso específico.

ARQUIVO	<code>backend/routes/depositos.routes.js</code>
LOCAL	<code>GET /empresa/exportar-dados</code>
AFETA WEB?	Sim, se a função for usada hoje.
AFETA ANDROID?	Só se o app expuser exportação de dados (não previsto nas fases iniciais).

MÉDIO**DELETE de conciliação bancária sem transação**

Remove os itens e depois a sessão de conciliação em duas instruções separadas, sem `BEGIN/COMMIT` — uma falha no meio deixa dados órfãos.

ARQUIVO	<code>backend/routes/conciliacao.routes.js</code>
LOCAL	<code>DELETE /conciliacao/:id</code>
AFETA ANDROID?	Só se conciliação bancária entrar no escopo mobile (não recomendado nas fases 1–8, ver Roadmap).

MÉDIO**Verificação de acesso multiempresa incompleta em atributos de grade**

Os quatro endpoints de `/grades/atributos` confiam apenas no `empresa_id` do próprio token, sem passar por `validarAcessoEmpresa()` como o resto do sistema faz — ainda seguro (o valor vem do JWT assinado, não do cliente), mas inconsistente com o padrão do restante da API.

ARQUIVO	<code>backend/routes/grades.routes.js</code>
IMPACTO	Baixo risco prático, alto valor de padronização.

BAIXO**Nome de tabela interpolado em rotas de lixeira (mitigado)**

`PUT/DELETE /lixeira/.../:tabela/:id` monta SQL com o nome da tabela diretamente na string — mas protegido por uma lista branca fixa (`produtos`, `clientes`, `fornecedores`) verificada antes de qualquer uso. Não é uma injeção explorável hoje; é um padrão a evitar em código novo.

ARQUIVO	<code>backend/routes/usuarios.routes.js</code>
---------	--

Painel superadmin SaaS é um "modo Deus" de plataforma — por desenho, não por falha

`admin.routes.js` e `admin-saas.routes.js` permitem que uma conta `is_saas_owner` liste, exporte e fature qualquer empresa cliente, sem qualquer isolamento multiempresa — o que é correto para uma equipe de operação de um SaaS, mas é a maior fronteira de confiança do sistema. Não é um bug; é um ponto a documentar formalmente (quem tem essa conta, como é auditada) antes de o app Android sequer considerar expor qualquer função administrativa cross-tenant — o que este documento **não recomenda** fazer no app do lojista.

B Mapa funcional completo

Inventário de todos os ≈ 340 endpoints encontrados nos 48 arquivos de rota, agrupados por domínio. Cada grupo abre para a lista completa endpoint a endpoint. Convenção da coluna **Android:** **Sim** vai na paridade inicial (Fases 2-6 do Roadmap), **Fase 2+** entra depois de o núcleo estar estável, **Consulta** começa só leitura, **N/A** é rota de plataforma/webhook sem UI. Salvo indicação contrária, toda rota passa por auth + validarAcessoEmpresa.

GRUPO NÚCLEO

100

auth, usuários, produtos, estoque, clientes, fornecedores, compras, vendas, grades, kits, depósitos, filiais, imagens, tabelas de preço

GRUPO FINANCEIRO

76

contas a pagar/receber, fluxo de caixa, caixa, conciliação, comissões, metas, devoluções, relatórios, dashboard, BI

GRUPO FISCAL & INTEGRAÇÕES

163

NF-e/NFC-e/NFS-e, checkout, pagamentos, webhooks, WhatsApp, marketplace, API pública, portal, CRM, fidelidade, admin SaaS

Autenticação & Sessão — auth.routes.js

9 rotas

FUNCIONALIDADE	ROTA	PERMISSÃO	ANDROID
Login	POST /login	público (rate limit)	Sim
Refresh de token	POST /auth/refresh	auth	Sim
Nonce para SSE	GET /auth/sse-token	auth	Fase 2+
Logout	POST /logout	auth	Sim
Editar perfil próprio	PUT /me/perfil	auth	Sim
Trocar senha própria	PUT /me/senha	auth	Sim
Histórico de acesso	GET /me/historico-acesso	auth	Fase 2+
Sessão atual	GET /me	auth	Sim
Status/limites da assinatura	GET /empresa/status	auth	Sim

Usuários, Permissões & Lixeira — usuarios.routes.js

9 rotas

Produtos, Grades, Kits, Imagens & Tabelas de Preço

38 rotas

Estoque, Depósitos & Rastreabilidade

22 rotas

Clientes & Fornecedores

11 rotas

Compras

3 rotas

Vendas & PDV

6 rotas

Financeiro — Contas a Pagar/Receber, Lançamentos, Fluxo de Caixa, Caixa, Conciliação

**40
rotas**

Comissões, Metas, Relatórios & BI

27 rotas

Fiscal — NF-e, NFC-e, NFS-e (Focus NFe)

20 rotas

Pagamentos — Checkout, PIX (EFÍ), Boletto (Asaas)

17 rotas

Webhooks, API Pública, Exportação Contábil

28 rotas

CRM, Fidelidade, Marketplace, WhatsApp, Alertas

39 rotas

Filiais, Portal do Cliente & Administração SaaS

23 rotas



Mapa de paridade Web → Android

Nenhuma funcionalidade desaparece. O que muda entre Web e Android é apenas quando cada uma entra — nunca se entra.

FUNCIONALIDADE WEB	EXISTE HOJE?	ANDROID	OBSERVAÇÃO
Login / sessão	Sim	Sim	Nativo, mesmo JWT; armazenamento em Keystore em vez de localStorage
Dashboard	Sim	Sim	Reorganizado por prioridade (Seção H), não apenas encolhido
PDV completo	Sim (com offline)	Sim	Prioridade máxima; ganha leitura de código de barras por câmera
Produtos, grades, kits, imagens, tabelas de preço	Sim	Sim	Upload de imagem usa câmera nativa
Estoque, depósitos	Sim	Sim	Ajuste e transferência incluídos
Rastreabilidade (lote/série)	Sim	Fase 2	Módulo especializado, baixo uso diário
Clientes, fornecedores	Sim	Sim	—
Compras (incl. importar XML de NF-e)	Sim	Sim	Mantida online (ver Seção I)
Vendas (histórico/edição)	Sim	Sim	—
Caixa (abertura/sangria/fechamento)	Sim	Sim	Fluxo diário do lojista, essencial no celular
Devoluções	Sim	Sim	—
Orçamentos / Pedidos	Sim	Sim	Inclui conversão em venda
Contas a receber / a pagar (baixa, parcial, estorno)	Sim	Sim	—

FUNCIONALIDADE WEB	EXISTE HOJE?	ANDROID	OBSERVAÇÃO
Lançamentos financeiros manuais	Sim	Sim	—
Fluxo de caixa (incl. projeção futura)	Sim	Sim	—
Investimentos, auditoria financeira	Sim	Fase 2	Uso esporádico
Conciliação bancária	Sim	Web por ora	Fluxo de importação de arquivo/matching é mais natural em tela grande; reavaliar
Comissões, Metas de vendas	Sim	Fase 2	Consulta é fácil; configuração pode esperar
Relatórios (resumo, CR, CP)	Sim	Consulta	Exportar/compartilhar via intents nativos
DRE, Lucratividade, Inadimplência, BI	Sim	Consulta	Gráficos adaptados, não a tabela densa da Web
Usuários e permissões	Sim	Sim	Sujeito a quem tem permissão de gerente/admin
Filiais	Sim	Sim	—
NF-e / NFC-e	Sim	Sim	Emissão pós-venda direto do PDV
NFS-e	Sim	Fase 2	Uso menos frequente (serviços)
PIX / Boleto / Links de pagamento	Sim	Sim	Já usado como forma de pagamento no PDV web
Fidelidade	Sim	Sim	Resgate integrado ao PDV
CRM, Marketplace, WhatsApp	Sim	Fase 2	Módulos de configuração/gestão, não de operação diária
Webhooks / API pública / Exportação contábil	Sim	Fase 2	Uso técnico/administrativo

FUNCIONALIDADE WEB	EXISTE HOJE?	ANDROID	OBSERVAÇÃO
Notificações (estoque baixo, vencimentos)	Sim (SSE na Web)	Sim	Vira push nativo (FCM) — ver Seção H
Painel SaaS (superadmin)	Sim	Fora de escopo	Ferramenta interna da operação LF ERP, não do lojista — ver achado de segurança
Portal do cliente final	Sim	Fora de escopo	É outro produto (para o cliente do lojista, não para o lojista)

Tecnologia recomendada

Kotlin + Jetpack Compose, nativo. Não WebView, não Flutter, não React Native. A decisão parte dos próprios requisitos que o sistema real impõe, não de preferência:

Por que não WebView

Rejeitado explicitamente pelo requisito do projeto — e tecnicamente errado aqui: o app precisa de câmera para leitura de código de barras (capacidade inexistente hoje na Web), armazenamento seguro em Keystore, e uma UX de toque que a SPA atual não foi desenhada para oferecer (ela é otimizada para mouse/teclado e telas largas).

Por que não React Native ou Flutter

O argumento clássico a favor de um framework híbrido é reaproveitar lógica já escrita em JavaScript. Isso não se aplica aqui: o frontend web é manipulação direta de DOM (sem camada de lógica de negócio isolada, reutilizável), e toda a regra de negócio real já vive no backend, atrás da API — não há nada em JS para "portar". Sobra então só o argumento de produtividade de UI, e mesmo esse enfraquece porque as duas capacidades mais críticas do app (câmera com leitura de código de barras em tempo real e armazenamento seguro com Android Keystore) exigem módulos nativos de qualquer forma em ambos os frameworks — o "escrever uma vez" não se realiza na prática para este produto.

Por que Kotlin + Compose

- **Câmera e código de barras:** CameraX + ML Kit Barcode Scanning (Google, on-device, gratuito) é a integração mais madura e rápida do ecossistema Android para exatamente o caso de uso do PDV (Seção 10 do briefing).
- **Segurança:** acesso direto ao Android Keystore/EncryptedSharedPreferences para o JWT, sem camada de ponte.
- **Offline:** Room + WorkManager reproduzem, com garantias mais fortes, o padrão que `pdvOffline.js` já validou em produção com IndexedDB.
- **Manutenção de longo prazo:** um único desenvolvedor/equipe pequena mantém hoje um backend Node e uma SPA vanilla — Kotlin nativo é uma segunda stack limpa e amplamente documentada, sem introduzir um terceiro runtime (Dart) ou a complexidade de bridges nativos do RN.

- **Publicação:** app nativo tem o caminho mais direto e previsível na Google Play, com melhor desempenho percebido e superfície de ataque menor.

DECISÃO A CONFIRMAR COM O CLIENTE

Este documento recomenda Kotlin/Compose com alta confiança dado o peso de câmera + segurança + offline no escopo. Se houver restrição orçamentária/tempo que favoreça reaproveitar algum desenvolvedor já fluente em Flutter, isso pode ser revisitado — mas não WebView e não um app que apenas embuta o site, por serem incompatíveis com os requisitos explícitos do projeto.

E Arquitetura do aplicativo

Clean Architecture + MVVM, com módulos por funcionalidade espelhando os domínios reais do backend (não uma divisão arbitrária) — cada módulo com suas camadas data / domain / presentation:

CAMADA	COMPONENTES	POR QUÊ
core/network	Retrofit + OkHttp + kotlinx.serialization, interceptor de Authorization, interceptor de refresh, mapeamento do envelope {sucesso, dados, erro}	Um único cliente HTTP para os ≈340 endpoints, nunca chamadas soltas em tela
core/security	EncryptedSharedPreferences/DataStore + Android Keystore	Token nunca em texto puro em disco
core/storage	Room (cache de produtos/clientes + fila de vendas offline)	Espelha IndexedDB do PDV web, com tipagem forte
core/sync	WorkManager (fila de sincronização com retry/backoff)	Reenvia vendas pendentes usando a mesma idempotency_key
core/errors	Mapeamento central de HTTP 400/401/403/404/409/422/429/500 → mensagem amigável	Espelha buildFriendlyError() do frontend web
feature/*	auth, dashboard, produtos, estoque, vendas+pdv, compras, financeiro, relatorios, usuarios, filiais, fiscal, pagamentos, configuracoes, notificacoes	Um módulo Gradle por domínio, igual ao agrupamento real das rotas — cada um pode evoluir e ser testado isoladamente

Pilha: Jetpack Compose + Navigation-Compose, Hilt (injeção de dependência), Coroutines/Flow em toda a camada assíncrona, Coil para imagens, CameraX + ML Kit para código de barras.

F Reuso de API

100% dos ≈ 340 endpoints mapeados na Seção B são reutilizáveis como estão — mesmo JWT, mesmo envelope de resposta, mesmas regras de `validarAcessoEmpresa` e `requirePermissao`. O app Android autentica exatamente como a SPA (`POST /login` → Bearer token), **não** através da camada `X-API-Key` de `/api/v1`, que existe para integrações de terceiros e não para o cliente oficial.

CORS não é um obstáculo: a lista de origens permitidas em `server.js` só se aplica a requisições de navegador — um cliente HTTP nativo (Retrofit/OkHttp) não envia `Origin` nem é bloqueado por essa política. Nenhuma alteração de CORS é necessária para o Android.

Mudanças necessárias no backend

Mínimas e aditivas — nenhuma reescrita, nenhuma rota nova obrigatória para começar:

1. **Nenhuma mudança em autenticação, permissões ou multiempresa.** O Android usa exatamente o que já existe.
2. **Verificar paginação** em cada endpoint de listagem que o app vai consumir (a maioria já aceita `limite/offset`) antes de implementar rolagem infinita — é investigação, não código novo.
3. **Opcional, aditivo:** um endpoint de registro de token push (POST `/usuarios/:id/fcm-token` ou equivalente) apenas se notificações push (Seção H) forem confirmadas para a Fase 2 — não afeta nenhuma rota existente.
4. **Opcional, avaliar depois de medir tráfego real:** uma rota de dashboard "resumida" com payload mais leve, só se o GET `/dashboard` atual se mostrar pesado demais em conexões móveis lentas — não implementar preventivamente.

Nenhuma alteração de schema é necessária para o offline de PDV: a coluna `idempotency_key` e o índice único parcial em vendas já existem (Seção A.4).

H UX/UI mobile

Navegação

Com ≈35 módulos reais, nenhuma estrutura única resolve tudo — a solução é hierárquica: **Bottom Navigation de 5 posições** (Início, PDV, Vendas, Financeiro, Mais) cobrindo os fluxos diários, com "Mais" abrindo uma grade/drawer com todos os módulos restantes agrupados exatamente como a Web já agrupa seu menu lateral (Operação, Comercial, Cadastros, Financeiro, Relatórios, Fiscal, Integrações, Configurações). PDV fica a um toque, conforme a prioridade da Seção 9 do briefing.

Dashboard mobile

Reorganizado, não encolhido: (1) KPIs essenciais em cards (faturamento, vendas, ticket médio), (2) alertas acionáveis (estoque baixo, contas vencendo), (3) um gráfico compacto expansível, (4) top produtos, (5) ações rápidas (nova venda, novo produto). DRE/BI avançado ficam em seções recolhíveis, não na primeira tela.

PDV — o fluxo mais crítico

Busca por texto ou câmera (CameraX + ML Kit) → grade de produto se aplicável → carrinho com quantidade/desconto por linha → cliente (opcional, com tabela de preço) → pagamento com split (Dinheiro, Pix, Cartão de Débito, Cartão de Crédito, Promissória — as mesmas cinco formas do PDV web) → confirmar → NFC-e opcional → recibo. Botão de finalizar bloqueado durante o envio para impedir duplo toque.

Formulários e valores

Teclado numérico para valores monetários com máscara BRL (R\$ 1.250,50), teclado apropriado para CPF/CNPJ/telefone, datas com seletor nativo. Toda soma/desconto/parcela é recalculada no servidor ao confirmar — o app nunca decide o valor final sozinho, exatamente como a Web já faz (tolerância de 0,05 verificada em POST /vendas).

I Estratégia offline

Seguro para cache (leitura)

Produtos, clientes, categorias, tabelas de preço, configurações — dados de referência, atualizados oportunisticamente via Room, exatamente o que `pdvOffline.js` já cacheia hoje em IndexedDB.

Seguro para fila offline (escrita) — apenas vendas do PDV

A venda é a única operação de escrita que este documento recomenda enfileirar offline, porque é a única que já tem a garantia de banco necessária: vendas possui `idempotency_key` com índice único parcial por empresa. O app gera a chave ao criar a venda (ainda offline), grava em Room, e o `WorkManager` reenvia ao reconectar — se a mesma chave já foi processada, o backend simplesmente devolve a venda existente em vez de duplicar. É o mesmo contrato que o PDV web já usa em produção.

MANTIDO ONLINE, DELIBERADAMENTE

Compras também tem `idempotency_key` no banco (tecnicamente poderia ser enfileirada), mas este documento recomenda mantê-la *online-only* na primeira fase — é uma operação menos frequente e menos sensível a tempo que uma venda no balcão, sem urgência que justifique o esforço de replicar a UI de itens no modo offline. **Baixas e estornos financeiros, ajustes manuais de estoque e devoluções não têm chave de idempotência no banco** — enfileirá-los offline hoje arriscaria duplicar um pagamento ou uma reversão de estoque. Ficam *online-only* até o backend ganhar o mesmo mecanismo para essas tabelas, se algum dia for necessário.

J Segurança

- **Token:** Android Keystore via EncryptedSharedPreferences/DataStore — mais rígido que o `localStorage/sessionStorage` que a própria Web usa hoje.
- **Transporte:** HTTPS obrigatório; sem pinning de certificado na v1 (o Render gerencia rotação de certificado; pinning ingênuo quebraria o app nesse cenário) — revisitar só se houver uma estratégia de rotação de pins.
- **Build:** ProGuard/R8 habilitado em release, sem segredo algum embutido no APK/AAB — a URL base muda por *build flavor* (dev/staging/prod), não por campo editável em runtime.
- **Telas sensíveis:** FLAG_SECURE em PDV/financeiro para bloquear captura de tela e prévia no app recente.
- **Bloqueio biométrico:** opcional, como camada extra de conveniência — nunca substitui o JWT.
- **Logs:** nunca token, senha ou payload de pagamento em log — mesma disciplina que o backend já aplica.

K Estratégia de testes

- **Unitários:** ViewModels, casos de uso e mapeadores de repositório (JUnit + Turbine para Flow).
- **Instrumentados/UI:** testes de Compose para os fluxos críticos — login, PDV completo (com e sem código de barras), baixa parcial, estorno, sincronização de venda offline.
- **Multiempresa e permissões, contra o backend real de staging:** tentar adulterar `empresa_id` numa requisição e confirmar 403; confirmar que um usuário sem permissão de módulo recebe 403 exatamente como a Web.
- **Regressão cruzada:** qualquer mudança de backend motivada pelo Android (Seção G) é validada contra a suíte Jest existente e manualmente contra a Web antes do deploy — nenhuma release quebra uma plataforma para consertar a outra.

Roadmap em fases

Adaptado às fases 1–10 do briefing, mapeadas para os módulos reais deste sistema. Nenhuma fase começa antes da aprovação deste documento.

1	Infraestrutura Projeto Kotlin/Compose, arquitetura de módulos, tema/design system, cliente de rede, Keystore, navegação base.
2	Base do sistema Login, sessão/refresh, dashboard reorganizado, menu, perfil.
3	Operação Produtos (+grades+kits+imagens+tabelas de preço), estoque, depósitos, fornecedores.
4	Compras Compras, itens, entrada automática de estoque, importação de XML de NF-e.
5	PDV — o build mais novo do projeto Carrinho, câmera + ML Kit para código de barras, split de pagamento, PIX, fila offline em Room/WorkManager, NFC-e.
6	Financeiro Contas a receber/pagar, baixas, parciais, estornos, lançamentos, caixa, fluxo de caixa.
7	Relatórios Resumo financeiro, DRE, lucratividade, inadimplência, BI (consulta + compartilhamento nativo).
8	Administração Usuários, permissões, filiais, configurações da empresa, fidelidade.
9	Fase 2 (pós-lançamento) Rastreabilidade, comissões, metas, CRM, marketplace, WhatsApp, NFS-e, conciliação bancária.

10

Qualidade & Release

Testes ponta a ponta, performance, segurança, compatibilidade de aparelhos, ícone/splash, assinatura, AAB, ficha da Google Play.

PRÓXIMO PASSO

Aguardando aprovação deste documento antes de iniciar a Fase 1. Nenhuma tela foi criada.